

PYTHON TRAINING NOTES

File Handling • Excel Files • Exception Handling • Modules & Packages • SQLite3 • Data Visualization

Purpose: Clean, class-style notes for learning, revision, interview preparation, and future project reference.

These notes reorganize the training topics into a logical sequence. Each topic contains: definition, why it is needed, syntax, commented Python examples, output/behavior, common mistakes, and project relevance.

1	File Handling in Python
2	Working with Excel Files
3	Error and Exception Handling
4	Modules, Libraries and Packages
5	SQLite3 with Python
6	Why Data Visualization Matters
7	Matplotlib, Seaborn, Plotly and Cufflinks
8	Important Charts and When to Use Them
9	Outliers, Box Plots and Heatmaps
10	Project Requirements and Best Practices

1. File Handling in Python

File handling means creating, opening, reading, writing, updating, copying, and deleting files using Python. It is important because real applications constantly work with files such as text files, CSV files, Excel files, logs, configuration files, uploaded documents, and reports.

Key idea: A program normally keeps variables only while it is running. Files allow information to be stored permanently and used again later.

1.1 Main File Modes

Mode	Meaning	Behavior
r	Read	Opens an existing file for reading. Error if the file does not exist.
w	Write	Creates a new file or overwrites all existing content.
a	Append	Adds new content at the end without deleting existing content.
x	Create	Creates a new file. Error if the file already exists.
b	Binary	Used with binary files such as images or PDFs, e.g. rb or wb.

1.2 Opening a File: Traditional Method

```
# Open the file in read mode
file = open("sample.txt", "r")

# Read the complete file content
content = file.read()

# Display the content
print(content)

# Always close the file after use
file.close()
```

Explanation: open() creates a file object. The mode 'r' means read. read() loads the content, and close() releases the file resource.

1.3 Recommended Method: with open(...)

```
# 'with' automatically closes the file after the block finishes
with open("sample.txt", "r") as file:
    # Read all text from the file
    content = file.read()

# The file is already closed here
print(content)
```

Key idea: Prefer with open(...). It automatically closes the file even if an error occurs, so it is safer and cleaner.

1.4 Writing to a File

```
# 'w' means write mode
# WARNING: Existing content will be deleted/overwritten
with open("notes.txt", "w") as file:
    # Write new text into the file
    file.write("Python file handling is easy to learn.\n")
    file.write("This is the second line.")

print("Data written successfully.")
```

What happens? If notes.txt already contains data, 'w' replaces it. If the file does not exist, Python creates it.

1.5 Appending to an Existing File

```
# 'a' means append mode
# Existing content is preserved
with open("notes.txt", "a") as file:
    # Add new content at the end
    file.write("\nThis line was added later.")

print("New content appended successfully.")
```

Use append mode when you want to add logs, new records, notes, or other content without destroying what is already stored.

1.6 Reading a Complete File, One Line, or All Lines

```
with open("notes.txt", "r") as file:
    # Reads the entire file as one string
    all_text = file.read()

print(all_text)

with open("notes.txt", "r") as file:
    # Reads only one line at a time
    first_line = file.readline()
    second_line = file.readline()

print(first_line)
print(second_line)

with open("notes.txt", "r") as file:
    # Reads all lines and returns a list
    lines = file.readlines()

# Index 0 means the first line
print(lines[0])

# Index 1 means the second line
print(lines[1])
```

Important distinction: `read()` returns one string, `readline()` returns the next single line, and `readlines()` returns a list of lines.

1.7 Reading Line by Line and Using `strip()`

```
with open("notes.txt", "r") as file:
    # Loop through the file one line at a time
    for line in file:
        # strip() removes leading/trailing spaces and newline characters
        clean_line = line.strip()
        print(clean_line)
```

`strip()` is useful because lines read from a text file often end with `\n`. It also removes unnecessary spaces from both ends.

1.8 Copying Content from One File to Another

```
# Open the source file for reading
with open("source.txt", "r") as source:
    # Read all content from the source
    content = source.read()

# Open the destination file for writing
with open("destination.txt", "w") as destination:
    # Copy the source content into the destination
    destination.write(content)

print("File content copied successfully.")
```

1.9 Inserting Text After a Specific Line

```
# Read all existing lines
with open("notes.txt", "r") as file:
    lines = file.readlines()
```

```
# Insert new text after line 2
# List index 2 means the new item becomes the third line
lines.insert(2, "This is the newly inserted line.\n")

# Rewrite the updated list of lines
with open("notes.txt", "w") as file:
    file.writelines(lines)
```

Key idea: Text files do not have a simple 'insert in the middle' operation. A common approach is: read the lines → modify the list → rewrite the file.

1.10 Checking Whether a File Is Closed

```
file = open("notes.txt", "r")

# False because the file is currently open
print(file.closed)

file.close()

# True because the file is now closed
print(file.closed)
```

1.11 Deleting a File

```
import os

file_name = "old_file.txt"

# Check first so the program does not crash
if os.path.exists(file_name):
    # remove() deletes the file from the system
    os.remove(file_name)
    print("File deleted.")
else:
    print("File does not exist.")
```

Correction to the class wording: `remove()` does not close a file. `file.close()` closes an open file. `os.remove()` deletes a file from the file system.

2. Working with Excel Files Using pandas

Python commonly uses the **pandas** library to organize tabular data. A pandas **DataFrame** is similar to an Excel table: rows represent observations and columns represent variables.

2.1 Creating a DataFrame

```
import pandas as pd

# Create data using a Python dictionary
data = {
    "Name": ["Anu", "Ravi", "Sara"],
    "Age": [24, 28, 26],
    "Score": [85, 91, 88]
}

# Convert the dictionary into a DataFrame
df = pd.DataFrame(data)

# Display the table
print(df)
```

Explanation: Each dictionary key becomes a column name. The lists become the values stored under those columns.

2.2 Saving a DataFrame to Excel

```
import pandas as pd

data = {
    "Name": ["Anu", "Ravi", "Sara"],
    "Score": [85, 91, 88]
}

df = pd.DataFrame(data)

# Save the DataFrame as an Excel file
# index=False prevents pandas from adding an extra row-number column
df.to_excel("students.xlsx", index=False)

print("Excel file created successfully.")
```

2.3 Reading Data from an Excel File

```
import pandas as pd

# Read the Excel file and convert it into a DataFrame
df = pd.read_excel("students.xlsx")

# Display the complete data
print(df)

# Display the first 5 rows
print(df.head())

# Check column names and missing values
print(df.info())
```

Key idea: For .xlsx files, pandas commonly uses the openpyxl engine. In a new environment, you may need to install it before reading or writing Excel files.

3. Error and Exception Handling

Exception handling prevents a program from stopping unexpectedly when a runtime problem occurs. Instead of crashing, the program can catch the error, display a useful message, perform cleanup, or continue safely.

3.1 The Four Main Blocks

Block	Purpose
try	Contains code that may produce an exception.
except	Runs when a matching exception occurs.
else	Runs only when the try block finishes without an exception.
finally	Runs regardless of whether an exception occurred.

3.2 FileNotFoundError Example

```
try:
    # Try to open a file that may not exist
    with open("missing_file.txt", "r") as file:
        content = file.read()

except FileNotFoundError:
    # This block runs if the file cannot be found
    print("The requested file was not found.")

else:
    # This runs only when no exception occurs
    print("File opened successfully.")
    print(content)

finally:
    # This always runs
    print("File operation completed.")
```

Flow: Python first executes try. If the file is missing, it jumps to except. If no error occurs, else executes. finally runs in both situations.

3.3 Handling Multiple Exception Types

```
try:
    # Ask the user for a number
    number = int(input("Enter a number: "))

    # This may raise ZeroDivisionError
    result = 100 / number

except ValueError:
    # Happens if the input cannot be converted to an integer
    print("Please enter a valid integer.")

except ZeroDivisionError:
    # Happens if the user enters 0
    print("Division by zero is not allowed.")

else:
    print("Result:", result)

finally:
    print("Program finished.")
```

Key idea: Catch specific exceptions whenever possible. A broad except: can hide real programming mistakes and make debugging harder.

4. Modules, Libraries and Packages

These terms are related but are often used loosely. The most useful practical distinction is below.

Term	Easy Meaning	Example
Module	Usually one Python .py file containing reusable code.	math, random, my_utils.py
Package	A directory that organizes multiple Python modules/subpackages.	A structured Python project/package
Library	A broad term for reusable code; may contain many modules and packages	pandas, NumPy

Key idea: A library is not strictly 'multiple .py files' by definition. It is a general term for reusable functionality. A package is the formal Python mechanism for organizing modules.

4.1 Built-in Module: random

```
import random

# randint includes both endpoints
# Possible result: any integer from 10 through 50
value = random.randint(10, 50)

print("Random number:", value)
```

4.2 Built-in Module: math

```
import math

number = 5

# factorial(5) = 5 × 4 × 3 × 2 × 1
result = math.factorial(number)

print("Factorial:", result)
```

4.3 Built-in Module: datetime

```
from datetime import datetime

# Get the current date and time
now = datetime.now()

# Extract only the current year
current_year = now.year

print("Current date and time:", now)
print("Current year:", current_year)
```

4.4 Creating a Custom Module

Suppose we create a file named my_utils.py:

```
# my_utils.py

def find_maximum(a, b, c):
    # Start by assuming a is the largest
    maximum = a

    # Replace maximum if b is larger
    if b > maximum:
        maximum = b

    # Replace maximum if c is larger
    if c > maximum:
        maximum = c

    # Return the final largest value
    return maximum
```

Now use that module from another Python file:

```
# main.py

# Import the custom module
import my_utils

# Call the function stored inside my_utils.py
answer = my_utils.find_maximum(10, 45, 23)

print("Maximum value:", answer)
```

The advantage is **reusability**. Instead of rewriting the same logic in every program, we write it once and import it wherever needed.

5. SQLite3 with Python

SQLite is a lightweight relational database stored in a local file. Python includes the sqlite3 module, so it is useful for learning SQL integration, desktop applications, prototypes, and smaller local applications.

5.1 Basic Database Workflow

The normal sequence is: **import** → **connect** → **create cursor** → **execute SQL** → **commit changes** → **fetch results** → **close connection**.

5.2 Create a Database and Table

```
import sqlite3

# Connect to a database file
# If the file does not exist, SQLite creates it
connection = sqlite3.connect("customers.db")

# Create a cursor object
# The cursor is used to execute SQL commands
cursor = connection.cursor()

# Create a table only if it does not already exist
cursor.execute("""
CREATE TABLE IF NOT EXISTS customer (
    customer_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    city TEXT
)
""")

# Save the database changes
connection.commit()

print("Customer table is ready.")

# Close the database connection
connection.close()
```

5.3 Inserting Data Safely

```
import sqlite3

connection = sqlite3.connect("customers.db")
cursor = connection.cursor()

# Use placeholders (?) instead of building SQL with string concatenation
# This is safer and helps prevent SQL injection
cursor.execute(
    "INSERT INTO customer (name, city) VALUES (?, ?)",
    ("Anu", "Memphis")
)

# INSERT changes the database, so commit the transaction
connection.commit()

print("Customer inserted successfully.")
```

```
connection.close()
```

Key idea: Use parameterized queries with ? placeholders. Do not insert user input directly into SQL strings.

5.4 Reading Data with SELECT

```
import sqlite3

connection = sqlite3.connect("customers.db")
cursor = connection.cursor()

# Execute a SELECT query
cursor.execute("SELECT * FROM customer")

# fetchall() returns all matching rows
rows = cursor.fetchall()

# Print each database row
for row in rows:
    print(row)

connection.close()
```

fetchone() returns one row, fetchmany(n) returns a limited number of rows, and fetchall() returns all remaining rows.

6. Data Visualization: Why It Matters

Large datasets are difficult to understand by looking only at rows and columns. Visualization converts data into charts and graphs so that patterns become easier to identify and communicate.

- **Distribution:** How values are spread across a range.
- **Trend:** Whether values increase, decrease, fluctuate, or show seasonality over time.
- **Comparison:** Which category, product, group, or model performs better.
- **Outliers:** Unusually high or low observations.
- **Relationship:** Whether two variables move together.
- **Missing-data patterns:** Where information is absent.
- **Business storytelling:** Turning analysis into information that decision-makers can understand.

Key idea: Visualization is not decoration. A chart should answer a specific question. Choosing a complicated 3D chart when a simple 2D chart communicates better is a mistake.

7. Main Python Visualization Libraries

Library	Best Use	Interaction
Matplotlib	Core plotting, full control, static 2D charts.	Mostly static
Seaborn	Statistical visualization built on Matplotlib.	Mostly static
Plotly	Interactive 2D/3D charts, hover, zoom, rotate.	Interactive
Cufflinks	Convenient Plotly integration with pandas DataFrames.	Interactive

7.1 Creating Sample Data

```
import numpy as np
import pandas as pd

# Set a seed so the same random values can be reproduced
np.random.seed(42)

# Generate 12 random sales values
sales = np.random.randint(50, 150, size=12)

# Create month numbers from 1 to 12
months = np.arange(1, 13)

# Build a DataFrame
df = pd.DataFrame({
    "Month": months,
    "Sales": sales
})

print(df)
```

Random sample data is useful for learning chart syntax. In real projects, replace it with your actual CSV, Excel, database, or API data.

8. Important Charts and When to Use Them

8.1 Line Chart — Trends Over Time

A line chart is best for continuous sequences, especially time. It helps identify upward trends, downward trends, spikes, drops, cycles, and seasonality.

```
import matplotlib.pyplot as plt
```

```
# Plot Month on the x-axis and Sales on the y-axis
plt.plot(df["Month"], df["Sales"], marker="o")

# Add chart labels
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")

# Display the chart
plt.show()
```

Business example: Monthly revenue, daily website traffic, yearly insurance claims, or model performance over training epochs.

8.2 Multiple Line Charts — Comparing Trends

```
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5, 6]
brand_a = [50, 60, 72, 80, 91, 105]
brand_b = [45, 58, 65, 77, 84, 95]

# Plot both series on the same axes
plt.plot(months, brand_a, marker="o", label="Brand A")
plt.plot(months, brand_b, marker="o", label="Brand B")

plt.title("Sales Trend Comparison")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

8.3 Bar Chart — Comparing Categories

Use a bar chart when the x-axis contains distinct categories. It makes side-by-side magnitude comparisons easy.

```
import matplotlib.pyplot as plt

brands = ["Samsung", "Apple", "Oppo", "Vivo"]
sales = [420, 510, 260, 230]

# Create a vertical bar chart
plt.bar(brands, sales)

plt.title("Annual Phone Sales by Brand")
plt.xlabel("Brand")
plt.ylabel("Units Sold")
plt.show()
```

8.4 Horizontal Bar Chart

```
import matplotlib.pyplot as plt

brands = ["Samsung", "Apple", "Oppo", "Vivo"]
sales = [420, 510, 260, 230]

# barh() creates horizontal bars
plt.barh(brands, sales)

plt.title("Annual Phone Sales by Brand")
plt.xlabel("Units Sold")
plt.ylabel("Brand")
plt.show()
```

Horizontal bars are often better when category names are long.

8.5 Stacked Bar Chart

A stacked bar chart shows a total while also showing how subcategories contribute to that total.

```
import matplotlib.pyplot as plt
```

```

months = ["Jan", "Feb", "Mar"]
product_a = [20, 25, 30]
product_b = [15, 18, 22]

# Plot the first category
plt.bar(months, product_a, label="Product A")

# Place Product B on top of Product A
plt.bar(months, product_b, bottom=product_a, label="Product B")

plt.title("Monthly Sales Composition")
plt.ylabel("Sales")
plt.legend()
plt.show()

```

Key idea: Stacked bars are useful for composition, but poor for precise comparison of the upper segments because they do not share the same baseline.

8.6 Pie Chart — Part-to-Whole Contribution

```

import matplotlib.pyplot as plt

categories = ["A", "B", "C", "D"]
values = [40, 30, 20, 10]

# autopct displays percentages
plt.pie(values, labels=categories, autopct="%1.1f%%")

plt.title("Percentage Contribution")
plt.show()

```

Use pie charts only when there are a small number of categories and the goal is simple part-to-whole communication. For many categories, a bar chart is usually clearer.

8.7 Scatter Plot — Relationship Between Two Numeric Variables

```

import matplotlib.pyplot as plt

advertising = [10, 20, 30, 40, 50, 60]
sales = [15, 25, 35, 42, 55, 63]

# Each point represents one observation
plt.scatter(advertising, sales)

plt.title("Advertising Spend vs Sales")
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.show()

```

Scatter plots help reveal positive relationships, negative relationships, clusters, non-linear patterns, and unusual points.

8.8 Box Plot — Distribution and Potential Outliers

A box plot summarizes the distribution using the median, lower quartile (Q1), upper quartile (Q3), interquartile range (IQR), whiskers, and points beyond the whiskers that may be potential outliers.

```

import matplotlib.pyplot as plt

values = [10, 12, 13, 14, 15, 16, 18, 19, 20, 55]

# The value 55 may appear as an outlier
plt.boxplot(values)

plt.title("Box Plot for Outlier Detection")
plt.ylabel("Value")
plt.show()

```

Important correction: In a standard Tukey box plot, the whiskers are not automatically the exact minimum and maximum. They commonly extend to the most extreme values within $1.5 \times \text{IQR}$. Values beyond that are plotted as

potential outliers.

Definitions: Q1 = 25th percentile, Median = 50th percentile, Q3 = 75th percentile, and IQR = Q3 – Q1.

8.9 Area Chart

An area chart is similar to a line chart, but the region under the line is filled. It emphasizes magnitude over time or cumulative contribution.

```
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5]
sales = [20, 35, 30, 50, 60]

# fill_between fills the area between the curve and baseline
plt.fill_between(months, sales, alpha=0.5)
plt.plot(months, sales)

plt.title("Sales Area Chart")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

Machine-learning connection: AUC means *Area Under the ROC Curve*. It is a numerical evaluation metric derived from the ROC curve; it is not simply the same thing as choosing an 'area chart' visualization.

8.10 Heatmap — Correlation Between Features

A correlation heatmap displays a correlation matrix using color. It is frequently used during exploratory data analysis to identify strongly related numeric features.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Age": [22, 25, 30, 35, 40],
    "Experience": [1, 3, 7, 12, 17],
    "Salary": [40000, 48000, 65000, 82000, 100000]
}

df = pd.DataFrame(data)

# Calculate pairwise correlations
correlation_matrix = df.corr(numeric_only=True)

# Plot the correlation matrix
sns.heatmap(
    correlation_matrix,
    annot=True,          # Show correlation numbers
    cmap="coolwarm",     # Color mapping
    vmin=-1,
    vmax=1
)

plt.title("Feature Correlation Heatmap")
plt.show()
```

Key idea: The meaning of a heatmap color depends on the selected color scale. Do not assume that red always means high correlation or blue always means low correlation. Read the color bar and the actual correlation values.

8.11 Andrews Curves

Andrews curves represent each multivariate observation as a curve. Similar observations may produce similar curve shapes, making the technique useful for exploratory comparison of multivariate classes.

```
import pandas as pd
import matplotlib.pyplot as plt
from pandas.plotting import andrews_curves
from sklearn.datasets import load_iris
```

```
# Load the Iris dataset
iris = load_iris(as_frame=True)
df = iris.frame

# Convert numeric target values into readable class names
df["species"] = df["target"].map(
    dict(enumerate(iris.target_names))
)

# Plot one curve per observation, grouped by species
andrews_curves(df.drop(columns=["target"]), "species")

plt.title("Andrews Curves - Iris Dataset")
plt.show()
```

Andrews curves are less common in day-to-day business dashboards. Use them when they genuinely help reveal multivariate grouping rather than simply because they look complex.

9. Interactive and 3D Visualization with Plotly

Plotly creates interactive charts. Users can hover over points, zoom, pan, select regions, and—in 3D charts—rotate the view. This can be valuable for exploration and web dashboards.

9.1 Interactive Line Chart

```
import plotly.express as px

# Create an interactive line chart directly from a DataFrame
fig = px.line(
    df,
    x="Month",
    y="Sales",
    title="Interactive Monthly Sales Trend",
    markers=True
)

# Open/display the interactive chart
fig.show()
```

9.2 Interactive 3D Scatter Plot

```
import plotly.express as px
import pandas as pd

data = pd.DataFrame({
    "Age": [22, 25, 30, 35, 40],
    "Income": [35000, 45000, 60000, 80000, 100000],
    "SpendingScore": [80, 70, 60, 45, 30],
    "Group": ["A", "A", "B", "B", "C"]
})

# x, y and z create the three dimensions
fig = px.scatter_3d(
    data,
    x="Age",
    y="Income",
    z="SpendingScore",
    color="Group",
    title="3D Customer Distribution"
)

fig.show()
```

A 3D scatter plot is useful when three numeric dimensions genuinely matter. However, 3D can also make exact comparison harder. Use it for exploration, not automatically for every dashboard.

9.3 3D Surface Plot

```
import numpy as np
import plotly.graph_objects as go

# Create x and y coordinates
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)

# Build a coordinate grid
X, Y = np.meshgrid(x, y)

# Calculate z values for the surface
Z = np.sin(np.sqrt(X**2 + Y**2))

# Create an interactive 3D surface
fig = go.Figure(
    data=[go.Surface(x=X, y=Y, z=Z)]
)

fig.update_layout(title="Interactive 3D Surface")
fig.show()
```

9.4 Bubble Chart

A bubble chart extends a scatter plot by using bubble size to represent an additional numeric variable.

```
import plotly.express as px
import pandas as pd

data = pd.DataFrame({
    "Income": [30, 45, 60, 80, 100],
    "Spending": [25, 40, 55, 70, 90],
    "Customers": [100, 180, 250, 320, 500],
    "Region": ["A", "B", "C", "D", "E"]
})

fig = px.scatter(
    data,
    x="Income",
    y="Spending",
    size="Customers", # Bubble size represents customer count
    hover_name="Region",
    title="Bubble Chart"
)

fig.show()
```

10. Outliers and Data Distribution

An outlier is an observation that is unusually far from most other observations. Outliers can be valid rare events, data-entry mistakes, fraud, unusual patients, exceptional sales, sensor failures, or other special cases.

- Do not automatically delete every outlier.
- First investigate why it exists.
- Check domain knowledge and data quality.
- Compare model performance with and without treatment.
- Possible treatments include correction, transformation, capping, robust models, or removal when justified.

Key idea: A rare value is not automatically a bad value. Removing legitimate extreme cases can destroy important information.

11. Choosing the Right Visualization

Question	Recommended Chart
How is a metric changing over time?	Line chart
Which category is larger?	Bar chart
What percentage does each category contribute?	Pie chart or 100% stacked bar
Are two numeric variables related?	Scatter plot
How is one numeric variable distributed?	Histogram / box plot
Are there potential outliers?	Box plot
How are numeric features correlated?	Correlation heatmap
How do multiple components contribute to a total?	Stacked bar / area chart
Do I need hover, zoom or 3D exploration?	Plotly interactive chart

12. Project Requirements — Practical Checklist

For your project, the goal should not be to add the maximum number of charts. The goal is to build a clear analysis that answers meaningful questions.

12.1 Data Understanding

- Explain the dataset and the business/problem statement.
- Identify rows, columns, target variables, numeric features, categorical features, and date/time fields.
- Check data types, duplicates, missing values, invalid values, and unusual records.

12.2 Exploratory Data Analysis (EDA)

- Use descriptive statistics to understand the data.
- Analyze distributions of important numeric features.
- Compare important categories.
- Study trends over time when a date/time variable exists.
- Investigate outliers instead of blindly removing them.
- Check correlations where relevant.

12.3 Visualization Requirements

- Include a meaningful mix of comparison, trend, distribution, relationship, and correlation visuals.
- Give every chart a clear title and labeled axes.
- Use readable legends when multiple series are present.
- Do not overload a single chart with too many categories or lines.
- Use interactive Plotly charts when hover, zoom, filtering, or 3D exploration genuinely adds value.
- Prefer simple 2D charts when they communicate the answer more clearly.

12.4 Business Insight

For every important chart, write a short conclusion:

- **What does the chart show?**
- **Why does it matter?**
- **What action or decision could follow?**

12.5 Example: Healthcare / Insurance

- How many claims occurred each month?
- Which claim categories have the highest total cost?
- Are there unusually expensive claims?
- Do some patients have repeated claims?
- Which health conditions are most common?
- How do claim patterns differ by age group, region, or other relevant categories?

A dashboard should turn these questions into visual evidence that a business user can understand quickly.

12.6 Recommended Project Structure

```
project/
├──
├── data/
│   └── dataset.csv
├──
├── notebooks/
│   └── analysis.ipynb
├──
├── outputs/
│   └── charts/
├──
├── src/
│   └── helper_functions.py
├──
├── requirements.txt
└── README.md
```

requirements.txt should list the external Python packages needed to run the project, for example:

```
pandas
numpy
matplotlib
seaborn
plotly
openpyxl
scikit-learn
```

13. Quick Revision Summary

- **File handling:** r reads, w overwrites/writes, a appends.
- **Best practice:** Use with open(...) so files close automatically.
- **Excel:** Use pandas DataFrames with read_excel() and to_excel().
- **Exceptions:** try → except → optional else → finally.
- **Module:** Reusable Python file; packages organize modules; library is a broader reusable-code term.
- **SQLite3:** connect → cursor → execute → commit/fetch → close.
- **Visualization:** Use charts to reveal trends, distributions, comparisons, relationships, correlations, and outliers.
- **Matplotlib/Seaborn:** Strong for static analysis; **Plotly:** strong for interactive charts.
- **Box plot:** Useful for distribution and potential outliers.
- **Heatmap:** Useful for correlation matrices; always interpret the scale.
- **Project rule:** Choose charts based on the question, not because a chart looks advanced.

14. Interview-Style Short Answers

What is file handling?

File handling is the process of creating, reading, writing, updating, and managing files programmatically.

Difference between write and append mode?

Write mode can overwrite existing content, while append mode adds new content at the end without deleting the existing data.

Why use with open()?

It automatically closes the file after the operation, including when an exception occurs.

What is exception handling?

Exception handling allows a program to catch runtime errors and respond safely instead of terminating unexpectedly.

What is a Python module?

A module is typically a Python file containing reusable functions, classes, or variables that can be imported into another program.

Why is `commit()` needed in `SQLite`?

It saves transaction changes such as INSERT, UPDATE, or DELETE permanently to the database.

Why do we use data visualization?

Data visualization makes patterns, trends, comparisons, relationships, and unusual observations easier to understand and communicate.

When would you use a box plot?

I use a box plot to understand a numeric distribution and identify potential outliers using quartiles and the interquartile range.

Why use `Plotly`?

Plotly is useful when interactive features such as hover details, zooming, filtering, or 3D exploration improve the analysis.

End of Notes